

- Now I can fully explain this architecture. This is the key design insight.

平台仓一致性原理: 模板引擎 + 数据配置分离

平台仓(dsar-hq-plat)之所以对所有车型保持一致,是因为它只提供“引擎”,不包含任何具体信号定义。具体信号由“数据配置”(从JSON + DBC 自动生成)注入,引擎和数据在同一次编译中被编进同一个 .a。

核心机制: 头文件契约 + 编译期绑定

平台仓 (所有车型相同)	
x_dom_can_rt_com.h → 定义通用结构体:	
can_msg_t, can_sig_base_info_t, can_msg_filter_item_t,	
SigHdl<phy, raw, factor, offset> 模板类	
sig_route_soc.cpp → 引擎代码 (引用生成配置中的 extern 表)	
调用: sig_route_send_mcu_tx_rx_sigs()	生成代码提供
调用: sig_route_update_write_to_mcu_msgs()	生成代码提供
引用: msg_route_mcu_tx_rx_msgs_map	生成代码提供
引用: msg_route_write_to_mcu_msgs_map	生成代码提供
x_dom_can_rt_soc.cpp → 初始化和Topic注册 (引用生成配置宏)	
引用: XDOMCANRT_DJI_COM_SERVICE_EN	生成代码提供
引用: XDOMCANRT_MSG_ROUTE_EN	生成代码提供
↑ 编译期接口(头文件契约)	
生成代码 (每个车型不同, 从JSON + DBC 生成)	
x_dom_can_rt_cfg.h → extern 声明 (所有具体信号和表)	
x_dom_can_rt_cfg.cpp → 实例化 (CanMsg对象, SigHdl对象, Map表)	
x_dom_can_rt_sigif.h → SigIf_Get/Set API 函数声明	
x_dom_can_rt_sigif.cpp → API 函数实现	

编译原理: 生成代码 + 平台引擎 → 同一个 .a

```
# 从 x_dom_can_rt_gen_com/CMakeLists.txt
aux_source_directory(${CMAKE_CURRENT_SOURCE_DIR} x_dom_can_rt_gen_src_list)
# ↑ 生成代码: x_dom_can_rt_cfg.cpp, x_dom_can_rt_sigif.cpp

aux_source_directory(${X_DOM_CAN_RT_SRC} x_dom_can_rt_sdk_src_list)
# ↑ 平台引擎: sig_route_soc.cpp, x_dom_can_rt_soc.cpp, msg_route_soc.cpp
# X_DOM_CAN_RT_SRC = ${CONAN_RES_DIRS_DSAR-PLAT-BF}/src/cdd/x_dom_can_rt

add_library(x_dom_can_rt_gen_lib_com_${PRODUCT_NAME} STATIC
    ${x_dom_can_rt_gen_src_list} # ← 车型特定的生成代码
    ${x_dom_can_rt_sdk_src_list}) # ← 所有车型相同的平台引擎
```

平台引擎源代码不是预编译的 shared library, 而是 .cpp 源码。每个车型编译时, 把平台通用引擎源码和本车型的生成配置源码一起编译, 链接成一个车型特定的静态库。这样:

- 引擎代码是 #include "x_dom_can_rt_cfg.h" 引用生成配置的
- 每个车型生成不同的 x_dom_can_rt_cfg.h, 引擎编译时就拿到不同的配置
- 最终产物是 x_dom_can_rt_gen_lib_com_fawhq_p301.a / x_dom_can_rt_gen_lib_com_fawhq_e001_10.a

生成代码提供的“接口实现”

平台引擎引用的、由生成代码提供的具体组件:

```
// ===== 1. 具体 CAN 消息对象 =====
// cfg.cpp 中为每个车型的每条CAN报文实例化一个 CanMsg
static CanMsg mcu_rx_msg_ABS_1_default_0xc0( // ← p301专属
    XDOMCANRT_CHANNEL_default, 0xc0, 32, 10); // 通道0, CAN ID 0xc0, DLC=32, 超时10ms
static CanMsg mcu_rx_msg_IBC_1_default_0xc1(...); // 同样处理IBC制动报文

// ===== 2. 信号处理器对象 =====
// 每个信号实例化一个 SigHdl, 模板参数是车型特定的 factor/offset
SigHdl<float, uint16_t, float, float> // phy=float, raw=uint16
    VehicleSpeed_ABS_1_default_mcu_rx_sighdl; // ← 车速信号
// 内部执行: phy = (raw * factor) + offset
// phy = (raw * 0.05625) + 0.0

SigHdl<uint8_t, uint8_t, uint8_t, uint8_t> // phy=uint8, raw=uint8
    IBC_BrakePedalStatus_IBC_1_default_mcu_rx_sighdl; // ← 制动踏板状态
// 内部执行: phy = (raw * 1) + 0 (直接透传)

// ===== 3. 信号路由表 (std::map) =====
const std::map<CanMsgIdPair, CanMsg*> msg_route_mcu_tx_rx_msgs_map = {
    {{0, 0x118}, &mcu_tx_msg_VDC_1_default_0x118}, // VDC纵向控制报文
    {{0, 0xc0}, &mcu_rx_msg_ABS_1_default_0xc0}, // ABS车速报文
    {{0, 0xc1}, &mcu_rx_msg_IBC_1_default_0xc1}, // IBC制动报文
    // ... 共31条报文的映射
};
```

平台引擎如何使用这些配置

以 sig_route_soc.cpp 为例, 引擎代码是 条件编译 的:

```
// 平台引擎 (所有车型相同)
#include "x_dom_can_rt_cfg.h" // ← 每个车型不同的配置

...
#ifdef XDOMCANRT_READ_MCU_RX_SIGS_EN // ← 这个宏来自生成的 cfg.h
// p301_150N中 read_mcu_rx_sigs 不为空 → 宏定义 → 这段会被编译
```

```

// p301 JSON中 read_mcu_rx_sigs 不为空 → 宏=1 → 这段会被编译
// 某个没有 read_mcu_rx_sigs 的车型 → 宏=0 → 这段直接跳过
void sig_route_send_mcu_tx_rx_sigs_proc(void) {
    sig_route_send_mcu_tx_rx_sigs(); // 调用生成代码中的函数
}
#endif
```

生成代码中的 `sig_route_send_mcu_tx_rx_sigs()` 遍历 `msg_route_mcu_tx_rx_msgs_map`，对每个信号调用对应的 `SigHdl` 对象的 `signal_if_get()` 将 raw → phy，然后通过 DCMS Topic 发送。

COM域 vs UDS域的区别

本质上是两条并行的生成管道，共享同一个平台引擎：

维度	COM域	UDS域
生成目录	<code>x_dom_can_rt_gen_com/</code>	<code>x_dom_can_rt_gen_uds/</code>
JSON 来源	<code>x_dom_can_rt_{product}.json</code> 的 route <code>dji_com_service</code>	同一个 JSON 的 route <code>dji_uds_service</code>
DCMS Topic 前缀	<code>DCMS_ADP_TOPIC_XDOMCANRT_COM_APP</code> (0x33100)	<code>DCMS_ADP_TOPIC_XDOMCANRT_UDS_APP</code> (0x33200)
编译产物	<code>x_dom_can_rt_gen_lib_com_{product}.a</code>	<code>x_dom_can_rt_gen_lib_uds_{product}.a</code>
链接目标	链接到 <code>{product}_com.so</code>	链接到 <code>{product}_uds_v4.so</code>
信号内容	车辆控制/状态信号 (车速、转角、制动等)	诊断相关CAN信号
平台引擎	相同的 <code>sig_route_soc.cpp</code> + <code>x_dom_can_rt_soc.cpp</code>	相同的 <code>sig_route_soc.cpp</code> + <code>x_dom_can_rt_soc.cpp</code>

它们都 `#include "x_dom_can_rt_cfg.h"` — 但因为编译时 `include` 路径指向不同目录(`gen_com` vs `gen_uds`)，所以各自看到不同的配置。同一份引擎代码，两套不同的数据，完全没有重复代码。

一句话总结

平台仓 = 通用数据结构 + 模板引擎(源码级复用)
生成代码 = 车型特定配置(JSON驱动, 自动生成)
编译时绑定: 引擎源码 + 生成配置 → 编译进同一个 .a

无论哪个车型、哪个域(COM/UDS)，平台仓代码完全不变。变的是编译时 `include` 的 `x_dom_can_rt_cfg.h`，这份头文件里 `extern` 声明的信号表、`SigHdl` 对象、`CanMsg` 对象，全都不一样，但都符合平台仓定义的结构体契约。